

Blockchain-ARP Benchmark Results

Run: 2026-08-28 17:25-21:07 UTC. Node counts 3,5,7,9. 500 trials each. Duration 3h42m (previous buggy run: 17h38m).

Bug found and fixed

benchmark.go used the raw trial number as the last IPv4 octet. Trials 256+ produced invalid addresses (e.g. 10.250.3.256) that were never detected. Result: exactly 245/500 (49%) timeouts in every latency/baseline file, always starting at trial 256. Fixed by wrapping the octet (trial/256, trial%256), same as coldstart already did. Pushed as commit 7dc6efd.

```
- ip := fmt.Sprintf("10.250.%d.%d", nodeCount, trial)
+ ip := fmt.Sprintf("10.250.%d.%d", nodeCount+trial/256, trial%256)

- ip := fmt.Sprintf("10.252.0.%d", trial)
+ ip := fmt.Sprintf("10.252.%d.%d", trial/256, trial%256)
```

Latency (detection_latency_ms)

Nodes	Trials	Mean	Median	P95	P99	Timeouts
3	500	1984.9	2012.5	2015.0	2016.0	0
5	500	1959.3	2013.0	2018.0	2019.0	0
7	500	1906.2	2012.0	2023.0	2025.0	0
9	500	1708.2	1635.0	2014.0	2033.0	0

Baseline (protected_detection_ms)

Nodes	Trials	Mean	Median	P95	P99
3	500	1987.0	2013.0	2015.0	2017.0
5	500	1951.2	2013.0	2020.0	2023.0
7	500	1774.8	1698.5	2029.0	2031.0
9	500	1669.5	1543.0	2037.0	2040.0

Coldstart (simulated_install_elapsed_ms by requested ledger size)

Nodes	Size 10	Size 50	Size 100	Size 250
3	1262	1347	1449	1756
5	3783	3783	3783	3783
7	4794	4791	4792	4791
9	5806	5806	5807	5805

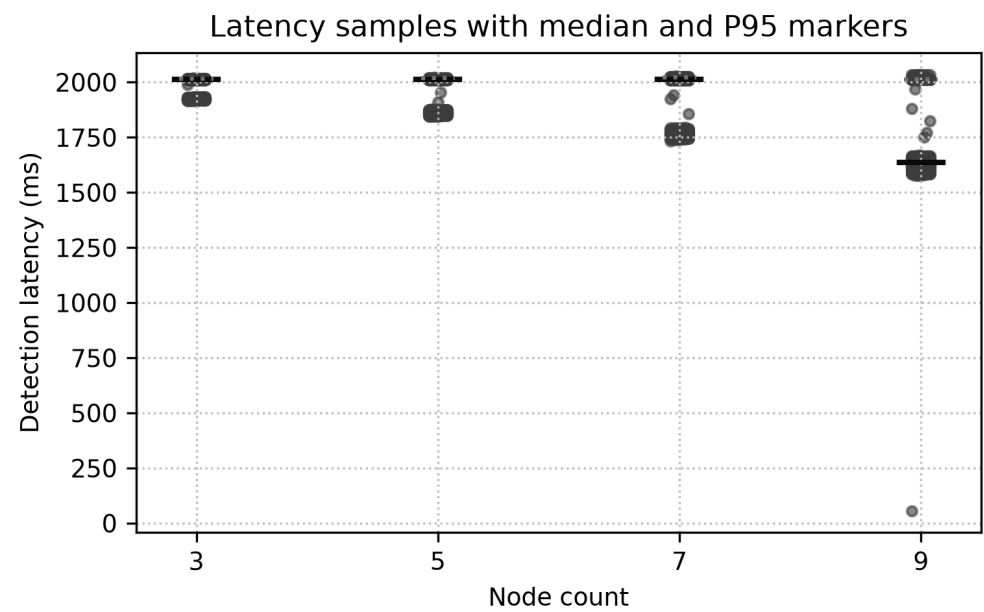
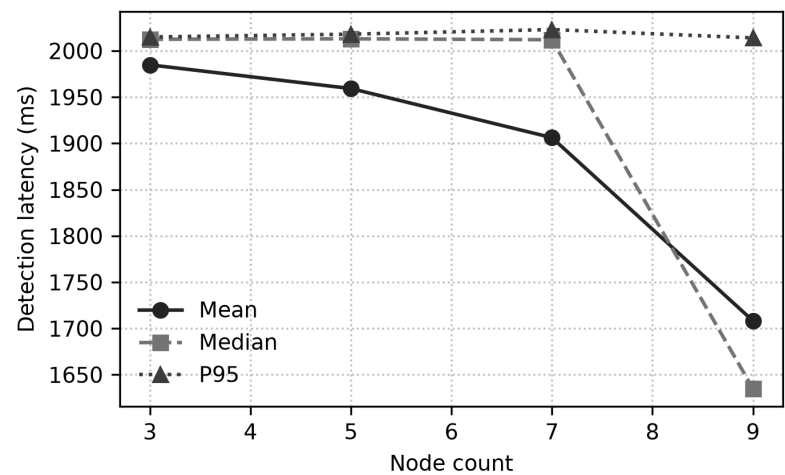
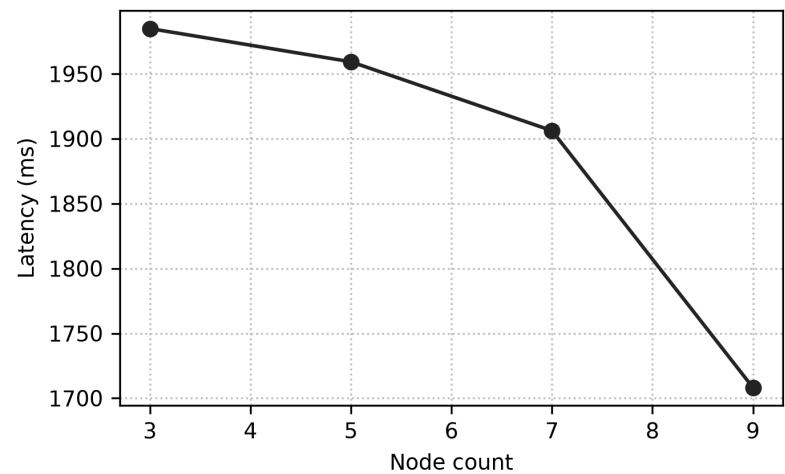
Note: flat across sizes within a node count because the ledger is never reset between node-count phases, so it keeps growing. The jump between node counts is real; the "vs. requested size" comparison within a node count is not meaningful.

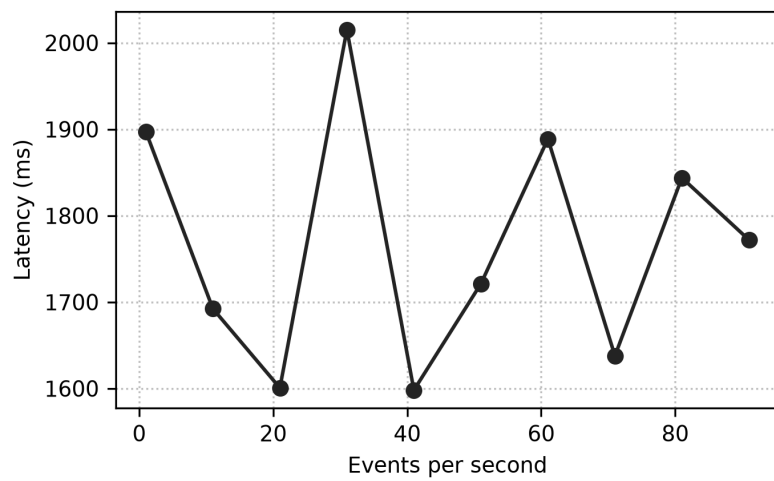
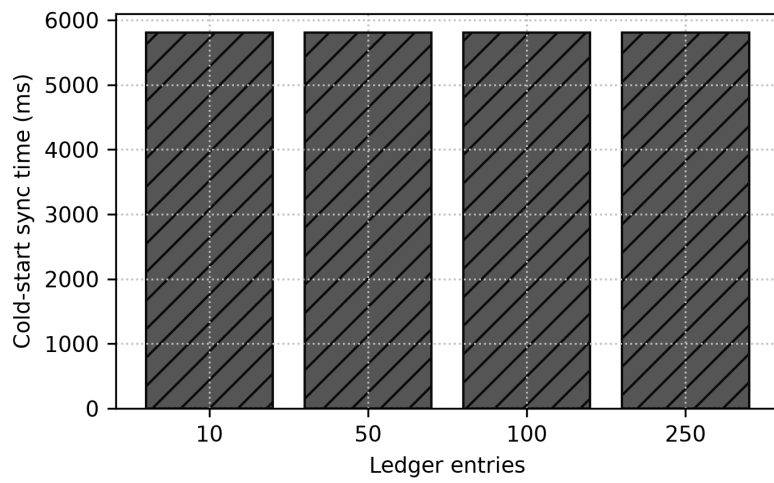
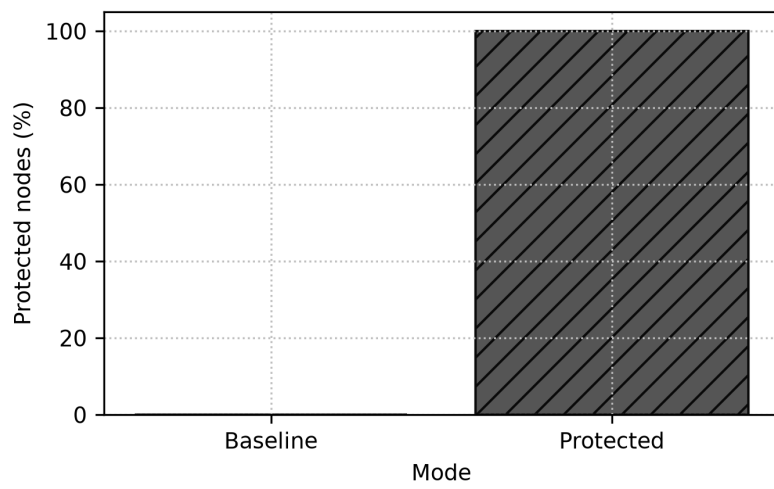
Throughput (detection_latency_ms by rate)

Nodes	Mean @ 1/s	Mean @ 91/s
3	1964.1	2012.5

5	1932.5	1914.2
7	1989.5	1800.5
9	1897.1	1772.0

Figures





Files

[benchmark/remote-results-20260828-fixed/](#)